



СБОРНИК ЗАДАНИЙ И РЕШЕНИЙ

Python — Часть 3: Профессиональный уровень

Генераторы • Декораторы • Алгоритмы • CSV/JSON • SQLite • Тестирование

☆ Лёгкие

★★ Средние

★★★ Сложные

□ **Совет:** В Части 3 задания сложнее — некоторые требуют установки библиотек (requests, sqlite3 встроен). Запускай каждый пример и экспериментируй с кодом!

Тема 15: Генераторы и итераторы

Генераторы экономят память — они выдают значения по одному, не создавая весь список сразу.

Задание 15.1: Бесконечный счётчик

☆☆☆

Напиши генератор `counter(start=0, step=1)`, который бесконечно выдаёт числа начиная с `start` с шагом `step`.

Возьми из него первые 5 значений через `next()`.

Потом создай генератор чётных чисел и выведи первые 10 через цикл с `break`.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>def counter(start=0, step=1):</code>	Параметры со значениями по умолчанию
<code> current = start</code>	Текущее значение счётчика
<code> while True:</code>	Бесконечный цикл — генератор не заканчивается
<code> yield current</code>	<code>yield</code> : отдаём значение и «засыпаем»
<code> current += step</code>	При следующем вызове продолжаем отсюда
<code># Первые 5 значений через next()</code>	
<code>gen = counter(10, 3)</code>	Начинаем с 10, шаг 3
<code>for _ in range(5):</code>	<code>_</code> — переменная-заглушка (значение не нужно)
<code> print(next(gen), end=' ')</code>	<code>next()</code> запрашивает следующее значение
<code>print()</code>	Переход на новую строку
<code># Чётные числа с break</code>	
<code>evens = counter(0, 2)</code>	Начало 0, шаг 2 → чётные
<code>count = 0</code>	Счётчик взятых значений
<code>for num in evens:</code>	<code>for</code> работает с генератором напрямую
<code> print(num, end=' ')</code>	
<code> count += 1</code>	
<code> if count >= 10:</code>	Остановиться после 10 чисел
<code> break</code>	<code>break</code> выходит из цикла

► Результат:

```
10 13 16 19 22
0 2 4 6 8 10 12 14 16 18
```

⊖ Частая ошибка:

✗ `def gen():` `yield 1` `yield 2 print(gen())`

✓ `for x in gen(): print(x)`

`gen()` возвращает объект-генератор, а не значения. Чтобы получить значения, используй `for`, `next()` или `list()`.

Задание 15.2: Генератор простых чисел

★★★

Напиши генератор `primes()`, который бесконечно выдаёт простые числа (2, 3, 5, 7, 11...).

Для проверки простоты используй вспомогательную функцию `is_prime(n)`.

Выведи первые 20 простых чисел.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>def is_prime(n):</code>	<i>Проверка числа на простоту</i>
<code> if n < 2:</code>	<i>Числа меньше 2 — не простые</i>
<code> return False</code>	
<code> for i in range(2, int(n**0.5) + 1):</code>	<i>Проверяем делители до $\sqrt{n}$</i>
<code> if n % i == 0:</code>	<i>Если делится без остатка — не простое</i>
<code> return False</code>	
<code> return True</code>	<i>Делителей нет → простое</i>
<code>def primes():</code>	<i>Генератор простых чисел</i>
<code> n = 2</code>	<i>Начинаем с 2 — первого простого</i>
<code> while True:</code>	<i>Бесконечно</i>
<code> if is_prime(n):</code>	<i>Проверяем текущее число</i>
<code> yield n</code>	<i>Отдаём простое число</i>
<code> n += 1</code>	<i>Переходим к следующему</i>
<code>gen = primes()</code>	<i>Создаём генератор</i>
<code>result = [next(gen) for _ in range(20)]</code>	<i>List comprehension + next() × 20</i>
<code>print(result)</code>	<i>Список первых 20 простых чисел</i>

► Результат:

```
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71]
```

Задание 15.3: Пайплайн из генераторов

★★★

Создай «конвейер» из трёх генераторов:

1. `read_numbers(lst)` — выдаёт числа из списка по одному
2. `filter_even(gen)` — пропускает только чётные
3. `square(gen)` — возводит в квадрат

Подключи их последовательно и выведи результат.

Входные данные: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>def read_numbers(lst):</code>	<i>Первый генератор — источник данных</i>
<code>for num in lst:</code>	<i>Перебираем список</i>
<code>yield num</code>	<i>Отдаём по одному числу</i>
<code>def filter_even(gen):</code>	<i>Второй генератор — фильтр</i>
<code>for num in gen:</code>	<i>Принимает другой генератор как источник</i>
<code>if num % 2 == 0:</code>	<i>Пропускаем нечётные</i>
<code>yield num</code>	<i>Отдаём только чётные</i>
<code>def square(gen):</code>	<i>Третий генератор — преобразование</i>
<code>for num in gen:</code>	
<code>yield num ** 2</code>	<i>Возводим каждое число в квадрат</i>
<code># Строим конвейер</code>	
<code>data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]</code>	<i>Исходные данные</i>
<code>stage1 = read_numbers(data)</code>	<i>→ 1,2,3,4,5,6,7,8,9,10</i>
<code>stage2 = filter_even(stage1)</code>	<i>→ 2,4,6,8,10</i>
<code>stage3 = square(stage2)</code>	<i>→ 4,16,36,64,100</i>
<code>print(list(stage3))</code>	<i>list() забирает все значения из генератора</i>
<code># Элегантный вариант в одну строку</code>	
<code>result =</code> <code>list(square(filter_even(read_numbers(data))))</code>	<i>Вложенный вызов — тот же конвейер</i>
<code>print(result)</code>	

► Результат:

```
[4, 16, 36, 64, 100]
[4, 16, 36, 64, 100]
```

□ **Мощь генераторов:** Конвейер обрабатывает данные по одному элементу — при миллионе чисел в памяти одновременно хранится только 1 элемент. `list()` в конце материализует результат.

□ Тест: Тема 15

? Вопрос: Что возвращает функция с yield?

А) Список значений

Б) Объект-генератор ← *правильный ответ*

В) Кортеж

Г) None

? Вопрос: Как получить следующее значение из генератора g?

А) g.next()

Б) next(g) ← *правильный ответ*

В) g.get()

Г) g()

? Вопрос: Чем генераторное выражение (x^2 for x in range(10)) отличается от $[x^2$ for x in range(10)]?

А) Ничем

Б) Генератор не хранит все значения в памяти ← *правильный ответ*

В) Генератор быстрее в 2 раза

Г) Генератор возвращает кортеж

Тема 16: Декораторы и замыкания

Декоратор — функция, которая оборачивает другую функцию, добавляя ей новое поведение без изменения кода.

Задание 16.1: Декоратор-таймер

☆☆☆

Напиши декоратор `@timer`, который:

- измеряет время выполнения функции
- выводит: «[имя_функции] выполнена за X.XXXX сек»

Примени его к функции `slow_sum(n)`, которая считает сумму `range(n)`.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import time</code>	Модуль для измерения времени
<code>from functools import wraps</code>	<code>wraps</code> сохраняет метаданные оригинальной функции
<code>def timer(func):</code>	Декоратор принимает функцию как аргумент
<code>@wraps(func)</code>	Сохраняем <code>__name__</code> , <code>__doc__</code> оригинала
<code>def wrapper(*args, **kwargs):</code>	<code>*args, **kwargs</code> — принимаем любые аргументы
<code>start = time.perf_counter()</code>	<code>perf_counter</code> — точнее чем <code>time()</code>
<code>result = func(*args, **kwargs)</code>	Вызываем оригинальную функцию
<code>end = time.perf_counter()</code>	Время после выполнения
<code>print(f"[{func.__name__}] выполнена за {end-start:.4f} сек")</code>	<code>func.__name__</code> — имя оборачиваемой функции
<code>return result</code>	Возвращаем результат оригинала
<code>return wrapper</code>	Возвращаем обёрнутую функцию
<code>@timer</code>	Синтаксический сахар: <code>slow_sum = timer(slow_sum)</code>
<code>def slow_sum(n):</code>	
<code>return sum(range(n))</code>	Сумма чисел от 0 до n-1
<code>print(slow_sum(1_000_000))</code>	<code>1_000_000 = 1000000</code> (удобная запись)
<code>print(slow_sum(10_000_000))</code>	10 миллионов — заметим разницу

► Результат:

```
[slow sum] выполнена за 0.0187 сек
499999500000
[slow sum] выполнена за 0.1923 сек
49999995000000
```

Задание 16.2: Декоратор с кэшированием

☆☆☆

Напиши декоратор `@cache`, который запоминает результаты вызовов функции. Если функция уже вызывалась с теми же аргументами — возвращает сохранённый результат.

Примени к рекурсивному `fibonacci(n)` и сравни скорость с кэшем и без.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import time</code>	
<code>def cache(func):</code>	<i>Декоратор-кэш</i>
<code> memo = {}</code>	<i>Словарь для хранения результатов</i>
<code> def wrapper(*args):</code>	<i>args — кортеж аргументов</i>
<code> if args not in memo:</code>	<i>Проверяем — уже считали с такими args?</i>
<code> memo[args] = func(*args)</code>	<i>Нет — считаем и сохраняем</i>
<code> return memo[args]</code>	<i>Возвращаем из кэша</i>
<code> return wrapper</code>	
<code>@cache</code>	<i>Применяем кэш к fibonacci</i>
<code>def fibonacci(n):</code>	
<code> if n <= 1:</code>	
<code> return n</code>	
<code> return fibonacci(n-1) + fibonacci(n-2)</code>	<i>Рекурсия — без кэша очень медленно</i>
<code># С кэшем</code>	
<code>start = time.perf_counter()</code>	
<code>print(fibonacci(40))</code>	<i>Без кэша это заняло бы минуты!</i>
<code>print(f"Время: {time.perf_counter() - start:.4f} сек")</code>	
<code># Python предлагает готовый кэш:</code>	
<code>from functools import lru_cache</code>	<i>Встроенный LRU-кэш</i>
<code>@lru_cache(maxsize=None)</code>	<i>maxsize=None — неограниченный кэш</i>
<code>def fib_fast(n):</code>	
<code> if n <= 1: return n</code>	
<code> return fib_fast(n-1) + fib_fast(n-2)</code>	
<code>print(fib_fast(100))</code>	<i>354224848179261915075 — мгновенно!</i>

► Результат:

```
102334155
Время: 0.0001 сек
354224848179261915075
```

⊖ Частая ошибка:

✗ `@timer def my_func():` `pass print(my_func.__name__) # 'wrapper'!`
 ✓ `from functools import wraps def timer(func):` `@wraps(func) #`
 добавить это! `def wrapper(...)`
 Без `@wraps` декоратор «ворует» имя у функции. `__name__` становится `'wrapper'`.
`@wraps(func)` сохраняет оригинальные метаданные.

□ Тест: Тема 16

? Вопрос: Что такое @decorator — синтаксический сахар для чего?

А) decorator.apply(func)

Б) func = decorator(func) ← *правильный ответ*

В) import decorator

Г) decorator + func

? Вопрос: Зачем нужен @wraps(func) внутри декоратора?

А) Ускоряет декоратор

Б) Сохраняет __name__ и __doc__ оригинала ← *правильный ответ*

В) Позволяет принимать аргументы

Г) Обязательный синтаксис

Тема 17: Алгоритмы сортировки и поиска

Алгоритмы — основа программирования. Понимание их работы поможет решать задачи эффективно.

Задание 17.1: Сортировка вставками

☆☆☆

Реализуй сортировку вставками (Insertion Sort).

Идея: берём каждый элемент и вставляем его на правильное место среди уже отсортированных.

Покажи состояние массива после каждого прохода.

Проверь на: [64, 25, 12, 22, 11]

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<pre>def insertion_sort(arr):</pre>	
<pre> arr = arr.copy()</pre>	<i>.copy() — не меняем оригинальный список</i>
<pre> n = len(arr)</pre>	
<pre> for i in range(1, n):</pre>	<i>Начинаем со второго элемента</i>
<pre> key = arr[i]</pre>	<i>Текущий элемент — «ключ» для вставки</i>
<pre> j = i - 1</pre>	<i>j — индекс последнего отсортированного</i>
<pre> while j >= 0 and arr[j] > key:</pre>	<i>Сдвигаем большие элементы вправо</i>
<pre> arr[j + 1] = arr[j]</pre>	<i>Сдвигаем элемент на одну позицию</i>
<pre> j -= 1</pre>	<i>Идём влево</i>
<pre> arr[j + 1] = key</pre>	<i>Вставляем ключ на найденное место</i>
<pre> print(f"Шаг {i}: {arr}")</pre>	<i>Показываем состояние после шага</i>
<pre> return arr</pre>	
<pre>data = [64, 25, 12, 22, 11]</pre>	<i>Тестовый массив</i>
<pre>print(f"До: {data}")</pre>	
<pre>result = insertion_sort(data)</pre>	
<pre>print(f"После: {result}")</pre>	

► Результат:

```
До:      [64, 25, 12, 22, 11]
Шаг 1: [25, 64, 12, 22, 11]
Шаг 2: [12, 25, 64, 22, 11]
Шаг 3: [12, 22, 25, 64, 11]
Шаг 4: [11, 12, 22, 25, 64]
После:  [11, 12, 22, 25, 64]
```

Задание 17.2: Бинарный поиск с логом

★★☆

Реализуй итеративный бинарный поиск.
Функция `binary_search(arr, target)` должна:

- возвращать индекс найденного элемента или -1
- выводить каждый шаг: `mid`-индекс, значение и решение (влево/вправо)
- считать количество сравнений

Проверь на отсортированном списке из 16 элементов.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>def binary_search(arr, target):</code>	
<code> left, right = 0, len(arr) - 1</code>	Начальные границы поиска
<code> steps = 0</code>	Счётчик шагов
<code> while left <= right:</code>	Пока есть область поиска
<code> mid = (left + right) // 2</code>	Середина текущей области
<code> steps += 1</code>	
<code> val = arr[mid]</code>	Значение в середине
<code> if val == target:</code>	Нашли!
<code> print(f"Шаг {steps}: arr[{mid}]= {val} — НАЙДЕН!")</code>	
<code> return mid, steps</code>	Возвращаем индекс и кол-во шагов
<code> elif val < target:</code>	Цель правее середины
<code> print(f"Шаг {steps}: arr[{mid}]= {val} < {target} → ищем правее")</code>	
<code> left = mid + 1</code>	Сдвигаем левую границу
<code> else:</code>	Цель левее
<code> print(f"Шаг {steps}: arr[{mid}]= {val} > {target} → ищем левее")</code>	
<code> right = mid - 1</code>	Сдвигаем правую границу
<code> return -1, steps</code>	Не нашли
<code>data = list(range(2, 34, 2))</code>	Чётные числа: [2,4,6,...,32]
<code>print(f"Массив: {data}")</code>	
<code>idx, n = binary_search(data, 22)</code>	Ищем 22
<code>print(f"Индекс: {idx}, шагов: {n}")</code>	

► Результат:

Массив: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32]
Шаг 1: `arr[7]=16 < 22` → ищем правее
Шаг 2: `arr[11]=24 > 22` → ищем левее
Шаг 3: `arr[9]=20 < 22` → ищем правее
Шаг 4: `arr[10]=22` — НАЙДЕН!
Индекс: 10, шагов: 4

□ **$O(\log N)$ в действии:** Для 16 элементов максимум 4 шага. Для 1 000 000 элементов — максимум 20 шагов! $\log_2(1000000) \approx 20$. Это в 50 000 раз быстрее линейного поиска.

□ Тест: Тема 17

? Вопрос: Какова сложность бинарного поиска?

А) $O(N)$

Б) $O(N^2)$

В) $O(\log N)$ ← *правильный ответ*

Г) $O(1)$

? Вопрос: Какое условие обязательно для бинарного поиска?

А) Список должен быть чётной длины

Б) Список должен быть отсортирован ← *правильный ответ*

В) Элементы должны быть числами

Г) Нет условий

Тема 18: Работа с CSV и JSON

CSV и JSON — самые популярные форматы для хранения и обмена структурированными данными.

Задание 18.1: Создание и чтение CSV

☆☆☆

Создай CSV-файл `grades.csv` с полями: имя, класс, математика, физика, история.
Добавь 5 записей.
Прочитай файл через `DictReader` и выведи:
— всех учеников с средним баллом > 80
— ученика с наивысшим средним баллом

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import csv</code>	Модуль для работы с CSV
<code># Данные</code>	
<code>students = [</code>	
<code> {"name": "Алёша", "class": "9А", "math": 95,</code>	
<code> "phys": 88, "hist": 72},</code>	
<code> {"name": "Маша", "class": "9Б", "math": 78,</code>	
<code> "phys": 92, "hist": 85},</code>	
<code> {"name": "Дима", "class": "9А", "math": 65,</code>	
<code> "phys": 70, "hist": 88},</code>	
<code> {"name": "Катя", "class": "9Б", "math": 91,</code>	
<code> "phys": 87, "hist": 94},</code>	
<code> {"name": "Вова", "class": "9А", "math": 55,</code>	
<code> "phys": 60, "hist": 72},</code>	
<code>]</code>	
<code># Запись</code>	
<code>with</code> <code>open("grades.csv", "w", newline="", encoding="utf-8") as f:</code>	<code>newline=""</code> важен на Windows
<code> fields =</code> <code> ["name", "class", "math", "phys", "hist"]</code>	Список полей для заголовка
<code> writer = csv.DictWriter(f,</code> <code> fieldnames=fields)</code>	<code>DictWriter</code> пишет из словарей
<code> writer.writeheader()</code>	Записывает строку-заголовок
<code> writer.writerows(students)</code>	Записывает все строки
<code># Чтение и анализ</code>	
<code>with open("grades.csv", "r", encoding="utf-8") as f:</code>	
<code> reader = csv.DictReader(f)</code>	<code>DictReader</code> читает в словари
<code> data = list(reader)</code>	Загружаем все строки в список
<code>for row in data:</code>	Вычисляем средний балл каждому

<code>avg = (int(row["math"]) + int(row["phys"]) + int(row["hist"])) / 3</code>	Значения из CSV — строки, нужен <code>int()</code>
<code>row["avg"] = round(avg, 1)</code>	Добавляем поле <code>avg</code>
<code>good = [r for r in data if r["avg"] > 80]</code>	Фильтр: средний > 80
<code>for r in good:</code>	
<code> print(f"{r['name']}:8 — средний {r['avg']}")</code>	
<code>best = max(data, key=lambda r: r["avg"])</code>	max с ключом по полю <code>avg</code>
<code>print(f"Лучший: {best['name']} ({best['avg']})")</code>	

► Результат:

Алёша — средний 85.0
 Маша — средний 85.0
 Катя — средний 90.7
 Лучший: Катя (90.7)

Задание 18.2: Конфигурация в JSON

★★★

Напиши класс `AppConfig`, который:

- хранит настройки приложения в словаре
- сохраняет их в JSON-файл `config.json`
- загружает при создании (если файл есть)
- имеет методы `get(key)`, `set(key, value)`, `reset()`

Поддерживаемые настройки: `theme`, `language`, `font_size`, `autosave`.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import json, os</code>	<code>json</code> для файлов, <code>os</code> для проверки существования
<code>class AppConfig:</code>	
<code> FILE = "config.json"</code>	Имя файла — константа класса
<code> DEFAULTS = {</code>	Значения по умолчанию
<code> "theme": "light",</code>	
<code> "language": "ru",</code>	
<code> "font_size": 14,</code>	
<code> "autosave": True</code>	
<code> }</code>	
<code> def __init__(self):</code>	
<code> if os.path.exists(self.FILE):</code>	Файл есть — загружаем
<code> with</code>	
<code>open(self.FILE, "r", encoding="utf-8") as f:</code>	
<code> self.data = json.load(f)</code>	<code>json.load</code> → Python-объект
<code> print("Настройки загружены")</code>	

else:	Файла нет — берём defaults
self.data = self.DEFAULTS.copy()	.copy() — не меняем оригинал
self._save()	Сохраняем defaults сразу
def _save(self):	Приватный метод сохранения
with open(self.FILE, "w", encoding="utf-8") as f:	
json.dump(self.data, f, ensure_ascii=False, indent=2)	ensure_ascii=False — русские символы
def get(self, key):	
return self.data.get(key)	.get() — безопасно, None если нет
def set(self, key, value):	
self.data[key] = value	Обновляем
self.save()	Сразу сохраняем на диск
print(f"Сохранено: {key} = {value}")	
def reset(self):	Сброс к значениям по умолчанию
self.data = self.DEFAULTS.copy()	
self._save()	
print("Настройки сброшены")	
cfg = AppConfig()	Создаём — загружает или создаёт файл
print(cfg.get("theme"))	light
cfg.set("theme", "dark")	Меняем тему
cfg.set("font_size", 16)	Меняем размер шрифта
print(cfg.get("font size"))	16

► **Результат:**

```
Настройки загружены (или созданы)
light
Сохранено: theme = dark
Сохранено: font_size = 16
16
```

□ Тест: Тема 18

? Вопрос: Что делает csv.DictReader?

- А) Записывает CSV из словарей
- Б) Читает CSV, возвращая строки как словари ← *правильный ответ*
- В) Конвертирует CSV в JSON
- Г) Сортирует CSV

? Вопрос: Какой параметр json.dump нужен для корректной записи русских букв?

- А) encoding='utf-8'
- Б) ensure_ascii=False ← *правильный ответ*
- В) allow_unicode=True
- Г) russian=True

Тема 19: Запросы к интернету (requests)

Библиотека requests — стандарт де-факто для HTTP-запросов в Python. Позволяет получать данные из любого API или веб-сервиса.

🔧 **Установка:** `pip install requests` — требуется только для этой темы, встроенного модуля нет.

Задание 19.1: Первый GET-запрос

☆☆☆

Сделай GET-запрос к публичному API: <https://httpbin.org/get>

Выведи:

- статус-код ответа
 - заголовок Content-Type
 - твой IP-адрес (он есть в ответе в поле origin)
 - время выполнения запроса
- Обработай возможные ошибки соединения.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import requests</code>	<i>pip install requests</i>
<code>import time</code>	<i>Для измерения времени</i>
<code>URL = 'https://httpbin.org/get'</code>	<i>httpbin.org — специальный сервис для тестирования HTTP</i>
<code>try:</code>	<i>Оборачиваем в try — запрос может упасть</i>
<code> start = time.perf_counter()</code>	<i>Засекаем время до запроса</i>
<code> response = requests.get(URL, timeout=10)</code>	<i>timeout=10 — не ждать дольше 10 секунд</i>
<code> elapsed = time.perf_counter() - start</code>	<i>Сколько времени заняло</i>
<code> print(f'Статус: {response.status_code}')</code>	<i>200 = ОК, 404 = не найдено, 500 = ошибка сервера</i>
<code> print(f'Content-Type: {response.headers["Content-Type"]}')'</code>	<i>headers — словарь заголовков ответа</i>
<code> data = response.json()</code>	<i>.json() парсит JSON-ответ в Python-объект</i>
<code> print(f'Мой IP: {data["origin"]}')'</code>	<i>origin — поле с IP-адресом клиента</i>
<code> print(f'Время: {elapsed:.3f} сек')</code>	
<code>except requests.exceptions.ConnectionError:</code>	<i>Нет интернета или сервер недоступен</i>
<code> print('Ошибка: нет соединения')</code>	
<code>except requests.exceptions.Timeout:</code>	<i>Сервер не ответил за timeout секунд</i>
<code> print('Ошибка: превышено время ожидания')</code>	
<code>except requests.exceptions.RequestException as e:</code>	<i>Любая другая ошибка requests</i>
<code> print(f'Ошибка запроса: {e}')</code>	

► **Результат:**

Статус: 200
Content-Type: application/json
Мой IP: 94.45.12.XXX
Время: 0.847 сек

⊖ **Частая ошибка:**

✗ `data = response.json print(data['origin'])`

✓ `data = response.json() print(data['origin'])`

`.json` — это метод, а не свойство. Без скобок `()` ты получишь объект метода, а не данные. Всегда пиши `response.json()` со скобками.

Задание 19.2: Работа с публичным API

★★★

Напиши программу для получения случайных фактов о числах.

API: <http://numbersapi.com/ЧИСЛО/trivia> (возвращает текст)

Пример: <http://numbersapi.com/42/trivia>

— попроси пользователя ввести число

— получи факт об этом числе

— получи математический факт: `/ЧИСЛО/math`

— получи факт о годе: `/ЧИСЛО/year`

Обработай ошибки и случай когда число некорректное.

✓ **РЕШЕНИЕ С ПОЯСНЕНИЯМИ**

```
import requests
```

```
BASE = 'http://numbersapi.com'
```

Базовый URL API

```
def get_fact(number, fact_type='trivia'):
```

fact_type по умолчанию trivia

```
    url = f'{BASE}/{number}/{fact_type}'
```

Формируем полный URL

```
    try:
```

```
        r = requests.get(url, timeout=8)
```

```
        r.raise_for_status()
```

Бросит исключение при 4xx/5xx

```
        return r.text
```

.text — ответ как строка (не JSON!)

```
    except requests.exceptions.HTTPError as e:
```

raise_for_status() вызывает HTTPError

```
        return f'Ошибка HTTP: {e}'
```

```
    except requests.exceptions.RequestException as e:
```

```
        return f'Ошибка: {e}'
```

```
def main():
```

```
    raw = input('Введи число: ').strip()
```

```
    try:
```

```
        n = int(raw)
```

Проверяем что ввели целое число

```
    except ValueError:
```

```
        print('Это не целое число!')
```

```
        return
```

Выходим из функции

```
    print(f'\n--- Факты о числе {n} ---')
```

types = [('trivia', 'Общий факт'),	
('math', 'Математика'),	
('year', 'Год')]	
for fact_type, label in types:	
fact = get_fact(n, fact_type)	
print(f'\n{label}:')	
print(f' {fact}')	
main()	

► Результат:

Введи число: 42

--- Факты о числе 42 ---

Общий факт:

42 is the answer to the Ultimate Question of Life, the Universe, and Everything.

Математика:

42 is a Catalan number.

Год:

42 is the year AD 42.

Задание 19.3: Мини-клиент для GitHub API

★★★

GitHub имеет открытый API. Напиши функции:

- get_user(username) — информация о пользователе
- get_repos(username) — список репозитория, отсортированных по звёздам
- get_repo_languages(username, repo) — языки конкретного репозитория

Выведи красиво отформатированный профиль.

API URL: <https://api.github.com/users/{username}>

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

import requests	
BASE = 'https://api.github.com'	Базовый URL GitHub API
HEADERS = {	
'Accept': 'application/vnd.github.v3+json',	Указываем версию API
'User-Agent': 'Python-Student-App'	GitHub требует User-Agent
}	
def api_get(url):	Вспомогательная функция для запросов
r = requests.get(url, headers=HEADERS, timeout=10)	
r.raise_for_status()	

return r.json()	
def get_user(username):	
return api_get(f'{BASE}/users/{username}')	Данные пользователя
def get_repos(username):	
repos = api_get(f'{BASE}/users/{username}/repos?per_page=100')	per_page=100 — до 100 репозиториев
return sorted(repos,	
key=lambda r: r['stargazers_count'],	Сортируем по звёздам
reverse=True)	По убыванию
def get_languages(username, repo):	
return api_get(f'{BASE}/repos/{username}/{repo}/languages')	Возвращает {язык: байт}
def print_profile(username):	
try:	
user = get_user(username)	
repos = get_repos(username)	
print(f'\n{'='*40}')	
print(f' {user["name"] or username}')	name может быть None
print(f' @{user["login"]}')	login — никнейм
print(f' {user["bio"] or "(нет описания)"}')	
print(f' Подписчиков: {user["followers"]}')	
print(f' Репозиториев: {user["public repos"]}')	
print(f{'='*40}')	
print(f'\nТоп-3 репозитория:')	
for repo in repos[:3]:	Только первые 3
stars = repo['stargazers_count']	
print(f' ★ {stars:5} {repo["name"]}')	★ звёзды — форматирование шириной 5
if repo['description']:	
print(f' {repo["description"][:60]}')	Первые 60 символов описания
except requests.exceptions.HTTPError as e:	
if '404' in str(e):	
print(f'Пользователь {username} не найден')	
else:	

<code>print(f'Ошибка: {e}')</code>	
<code>print_profile('torvalds')</code>	<i>Линус Торвальдс — создатель Linux</i>

► **Результат:**

```
=====
Linus Torvalds
@torvalds
Linux and Git
Подписчиков: 230000+
Репозиторий: 8
=====

Топ-3 репозитория:
★ 220000 linux
    Linux kernel source tree
★ 5000 ueemacs
    ...
...
```

□ **Лимит запросов:** GitHub API без авторизации позволяет 60 запросов в час. Для большего числа запросов нужен Personal Access Token. Добавь в headers: 'Authorization': 'token ВАШ_ТОКЕН'

? Вопрос: Что возвращает `response.json()`?

А) Строку с JSON

Б) Python-объект (dict/list) ← *правильный ответ*

В) Байты

Г) XML

? Вопрос: Для чего нужен параметр `timeout` в `requests.get()`?

А) Задерживает запрос намеренно

Б) Ограничивает время ожидания ответа ← *правильный ответ*

В) Количество повторов

Г) Размер ответа

? Вопрос: Что делает `response.raise_for_status()`?

А) Выводит статус-код

Б) Бросает исключение если статус не 2xx ← *правильный ответ*

В) Повторяет запрос при ошибке

Г) Логирует статус

Тема 20: Базы данных SQLite

SQLite — встроенная база данных. Не нужно ничего устанавливать. Данные хранятся в одном файле.

Задание 20.1: Первая база данных

☆☆☆

Создай базу данных library.db с таблицей books:

— id (PRIMARY KEY AUTOINCREMENT)

— title (TEXT)

— author (TEXT)

— year (INTEGER)

— available (INTEGER, 0 или 1)

Добавь 4 книги. Выведи все книги. Выведи только доступные.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import sqlite3</code>	<i>Встроенный модуль, ничего устанавливать не надо</i>
<code>conn = sqlite3.connect("library.db")</code>	<i>Открываем/создаём файл БД</i>
<code>conn.row_factory = sqlite3.Row</code>	<i>Строки будут как словари (удобнее)</i>
<code>cur = conn.cursor()</code>	<i>Курсор — инструмент для выполнения SQL</i>
<code># Создаём таблицу</code>	
<code>cur.execute('''</code>	<i>Тройные кавычки — многострочная строка</i>
<code>CREATE TABLE IF NOT EXISTS books (</code>	<i>IF NOT EXISTS — не падать если уже есть</i>
<code>id INTEGER PRIMARY KEY</code>	<i>AUTOINCREMENT — ID назначается сам</i>
<code>AUTOINCREMENT,</code>	
<code>title TEXT NOT NULL,</code>	<i>NOT NULL — поле обязательно</i>
<code>author TEXT NOT NULL,</code>	
<code>year INTEGER,</code>	
<code>available INTEGER DEFAULT 1</code>	<i>DEFAULT 1 — по умолчанию доступна</i>
<code>)</code>	
<code>''')</code>	
<code># Добавляем книги</code>	
<code>books = [</code>	
<code>("Мастер и Маргарита", "Булгаков",</code>	
<code>1967, 1),</code>	
<code>("1984", "Оруэлл", 1949,</code>	
<code>1),</code>	
<code>("Дюна", "Герберт", 1965,</code>	<i>0 — книга недоступна (выдана)</i>
<code>0),</code>	
<code>("Пикник на обочине",</code>	
<code>"Стругацкие", 1972, 1),</code>	
<code>]</code>	

<code>cur.executemany(</code>	<i>executemany — вставляем несколько строк</i>
<code> 'INSERT INTO books (title,author,year,available) VALUES (?,?,?,?) ',</code>	<i>? — плейсхолдеры, защита от SQL-инъекций</i>
<code> books</code>	
<code>)</code>	
<code>conn.commit()</code>	<i>ОБЯЗАТЕЛЬНО: сохраняем изменения</i>
<code># Читаем все книги</code>	
<code>print("Все книги:")</code>	
<code>for row in cur.execute("SELECT * FROM books"):</code>	<i>SELECT * — все столбцы</i>
<code> status = "✓" if row["available"] else "X"</code>	<i>Иконка статуса</i>
<code> print(f" [{status}] {row['title']} — {row['author']}, {row['year']}")</code>	
<code># Только доступные</code>	
<code>print("Доступные:")</code>	
<code>for row in cur.execute("SELECT * FROM books WHERE available=1"):</code>	<i>WHERE фильтрует строки</i>
<code> print(f" {row['title']}")</code>	
<code>conn.close()</code>	<i>Закрываем соединение</i>

► Результат:

Все книги:

```
[✓] Мастер и Маргарита — Булгаков, 1967
[✓] 1984 — Оруэлл, 1949
[X] Дюна — Герберт, 1965
[✓] Пикник на обочине — Стругацкие, 1972
```

Доступные:

```
Мастер и Маргарита
1984
Пикник на обочине
```

Задание 20.2: CRUD-приложение

★★★

Создай класс NoteDB — записная книжка с SQLite:

- `add(title, text)` — добавить заметку
- `get_all()` — все заметки
- `search(keyword)` — поиск по тексту
- `delete(note_id)` — удалить
- `update(note_id, new_text)` — обновить

Используй `contextmanager` для управления соединением.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

import sqlite3	
from contextlib import contextmanager	<i>contextmanager для with-блоков</i>
from datetime import datetime	<i>Для временных меток</i>
class NoteDB:	
def __init__(self, path="notes.db"):	
self.path = path	
self.init()	<i>Создаём таблицу при старте</i>
@contextmanager	<i>Декоратор превращает функцию в менеджер контекста</i>
def _conn(self):	
c = sqlite3.connect(self.path)	
c.row_factory = sqlite3.Row	<i>Строки как словари</i>
try:	
yield c	<i>Отдаём соединение в with-блок</i>
c.commit()	<i>Автоматический commit при успехе</i>
except:	
c.rollback()	<i>Откат при ошибке</i>
raise	<i>Пробрасываем ошибку дальше</i>
finally:	
c.close()	<i>Всегда закрываем</i>
def _init(self):	
with self._conn() as c:	<i>Используем наш менеджер</i>
c.execute('''CREATE TABLE IF NOT EXISTS notes(	
id INTEGER PRIMARY KEY	
AUTOINCREMENT,	
title TEXT NOT NULL,	
text TEXT,	
created TEXT	<i>Дата создания</i>
)''')	
def add(self, title, text):	
now =	<i>Форматируем дату</i>
datetime.now().strftime("%d.%m.%Y %H:%M")	
with self._conn() as c:	
c.execute("INSERT INTO	<i>? защищают от SQL-инъекций</i>
notes(title,text,created) VALUES(?,?,?)",	
(title, text, now))	
print(f'Добавлено: "{title}"')	
def get_all(self):	
with self._conn() as c:	

<pre> return c.execute("SELECT * FROM notes ORDER BY id DESC").fetchall() </pre>	<i>fetchall() — все строки сразу</i>
<pre> def search(self, kw): with self._conn() as c: return c.execute("SELECT * FROM notes WHERE text LIKE ? OR title LIKE ?", (f"%{kw}%", f"%{kw}%")).fetchall() </pre>	<i>LIKE — поиск с подстрокой</i> <i>% — любые символы вокруг ключевого слова</i>
<pre> def delete(self, note_id): with self._conn() as c: c.execute("DELETE FROM notes WHERE id=?", (note_id,)) </pre>	<i>(note_id,) — кортеж из одного элемента</i>
# Тест	
db = NoteDB()	
db.add("Python", "Генераторы экономят память")	
db.add("SQL", "SELECT * FROM table")	
db.add("Git", "git commit -m 'message'")	
<pre> for note in db.get_all(): print(f"#{note['id']} {note['title']}: {note['text'][:30]}") </pre>	<i>Первые 30 символов текста</i>
print("Поиск 'git':")	
<pre> for n in db.search("git"): print(f" Найдено: {n['title']}") </pre>	<i>Ищем по слову «git»</i>

► **Результат:**

```

#3 Git: git commit -m 'message'
#2 SQL: SELECT * FROM table
#1 Python: Генераторы экономят память
Поиск 'git':
    Найдено: Git

```

□ Тест: Тема 20

? Вопрос: Зачем использовать ? в SQL-запросах вместо f-строк?

A) Это быстрее

Б) Защита от SQL-инъекций ← *правильный ответ*

В) Поддержка Unicode

Г) Только синтаксическое требование

? Вопрос: Что делает conn.commit()?

А) Закрывает соединение

Б) Сохраняет изменения в БД ← *правильный ответ*

В) Начинает транзакцию

Г) Откатывает изменения

Тема 21: Многопоточность

Потоки позволяют выполнять несколько задач «одновременно». Особенно полезно когда программа ждёт сеть или файлы.

Задание 21.1: Параллельный отсчёт

☆☆☆

Напиши программу, которая запускает 3 потока одновременно.
Каждый поток считает от 1 до 5 с паузой 0.3 сек и выводит:
«Поток N: значение X»
Покажи разницу — сначала последовательно (все вместе ~4.5 сек),
потом параллельно (все вместе ~1.5 сек).
Измерь время обоих вариантов.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import threading</code>	<i>Встроенный модуль для потоков</i>
<code>import time</code>	
<code>def count(thread_num, steps=5, delay=0.3):</code>	<i>Функция для каждого потока</i>
<code>for i in range(1, steps + 1):</code>	
<code>time.sleep(delay)</code>	<i>Имитируем работу (ожидание)</i>
<code>print(f'Поток {thread_num}: шаг {i}')</code>	
<code># 1. Последовательно</code>	
<code>print('--- Последовательно ---')</code>	
<code>start = time.perf_counter()</code>	
<code>count(1)</code>	<i>Ждём завершения первого</i>
<code>count(2)</code>	<i>Потом запускаем второй</i>
<code>count(3)</code>	<i>Потом третий</i>
<code>seq_time = time.perf_counter() - start</code>	
<code>print(f'Последовательно: {seq_time:.2f} сек\n')</code>	<i>~4.5 сек (3 потока × 5 шагов × 0.3 сек)</i>
<code># 2. Параллельно</code>	
<code>print('--- Параллельно ---')</code>	
<code>start = time.perf_counter()</code>	
<code>threads = [</code>	<i>Создаём три потока</i>
<code>threading.Thread(target=count,</code> <code>args=(1,)),</code>	<i>target — функция, args — аргументы кортежем</i>
<code>threading.Thread(target=count,</code> <code>args=(2,)),</code>	<i>Запятая обязательна — это кортеж из 1 элемента</i>
<code>threading.Thread(target=count,</code> <code>args=(3,)),</code>	
<code>]</code>	

<code>for t in threads:</code>	<i>Запускаем все потоки</i>
<code>t.start()</code>	<i>.start() — поток начинает выполняться</i>
<code>for t in threads:</code>	<i>Ждём завершения всех потоков</i>
<code>t.join()</code>	<i>.join() — блокирует пока поток не закончит</i>
<code>par_time = time.perf_counter() - start</code>	
<code>print(f'\nПараллельно: {par_time:.2f} сек')</code>	<i>~1.5 сек — в 3 раза быстрее!</i>
<code>print(f'Ускорение: {seq_time/par_time:.1f}x')</code>	<i>Во сколько раз быстрее</i>

► Результат:

```

--- Последовательно ---
Поток 1: шаг 1
Поток 1: шаг 2
... (всё подряд) ...
Последовательно: 4.51 сек

--- Параллельно ---
Поток 1: шаг 1
Поток 3: шаг 1
Поток 2: шаг 1
... (вперемешку) ...
Параллельно: 1.52 сек
Ускорение: 3.0x

```

⊖ Частая ошибка:

✗ `t = threading.Thread(target=count(1))` # сразу вызывает!

✓ `t = threading.Thread(target=count, args=(1,))` # передаём функцию

target=count(1) немедленно вызывает count(1) и передаёт результат (None) как target. Нужно передавать саму функцию: target=count, а аргументы отдельно через args.

Задание 21.2: ThreadPoolExecutor: массовая загрузка

★★★

Используй ThreadPoolExecutor для параллельной загрузки данных.
 Дан список из 5 URL (httpbin.org/delay/1 — отвечает с задержкой 1 сек).
 Загрузи все последовательно — замерь время.
 Загрузи все параллельно — замерь время.
 Обработай ошибки для каждого URL отдельно.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

```

import requests
import time

```

from concurrent.futures import ThreadPoolExecutor, as_completed	Современный способ работы с потоками
URLS = ['https://httpbin.org/delay/1', 'https://httpbin.org/delay/1', 'https://httpbin.org/delay/1', 'https://httpbin.org/delay/1', 'https://httpbin.org/delay/1',]	5 URL с искусственной задержкой 1 сек Каждый отвечает через ~1 секунду
def fetch(url):	Функция загрузки одного URL
try:	
r = requests.get(url, timeout=15)	
return url, r.status_code, None	Возвращаем (url, статус, ошибка)
except Exception as e:	
return url, None, str(e)	При ошибке — передаём текст
# 1. Последовательная загрузка	
print('Последовательная загрузка...')	
start = time.perf_counter()	
for url in URLS:	
url, code, err = fetch(url)	Ждём каждый по очереди
print(f' {code or err}')	
seq = time.perf_counter() - start	
print(f'Время: {seq:.2f} сек')	~5 сек
# 2. Параллельная загрузка	
print('\nПараллельная загрузка...')	
start = time.perf_counter()	
with ThreadPoolExecutor(max_workers=5) as executor:	max_workers — сколько потоков
futures = {executor.submit(fetch, url): url for url in URLS}	submit() отправляет задачу, возвращает Future
	futures — словарь future: url
for future in as_completed(futures):	as_completed — итератор по завершённым
url, code, err = future.result()	.result() — получаем результат
print(f' {code or err}: {url[:40]}')	
par = time.perf_counter() - start	
print(f'Время: {par:.2f} сек')	~1 сек — в 5 раз быстрее!
print(f'Ускорение: {seq/par:.1f}x')	

► Результат:

```
Последовательная загрузка...
200
200
200
```

```
200
200
Время: 5.23 сек
```

```
Параллельная загрузка...
200: https://httpbin.org/delay/1
200: https://httpbin.org/delay/1
200: https://httpbin.org/delay/1
200: https://httpbin.org/delay/1
200: https://httpbin.org/delay/1
Время: 1.18 сек
Ускорение: 4.4x
```

Задание 21.3: Поток-демон и Lock

★★★

Создай программу с двумя потоками:

- Поток-«производитель»: каждые 0.5 сек генерирует случайное число и добавляет в общий список
- Поток-«потребитель»: каждые 1 сек берёт все числа из списка, считает сумму и выводит

Используй `threading.Lock()` чтобы защитить общий список от одновременного доступа. Запусти на 5 секунд, потом останови.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

```
import threading
import random
import time
```

<code>shared_data = []</code>	<i>Общий список — оба потока его используют</i>
<code>lock = threading.Lock()</code>	<i>Lock — «замок» для синхронизации</i>
<code>stop_event = threading.Event()</code>	<i>Event — сигнал для остановки потоков</i>
<code>def producer():</code>	<i>Поток-производитель</i>
<code>while not stop_event.is_set():</code>	<i>.is_set() — True если событие произошло</i>
<code>num = random.randint(1, 100)</code>	<i>Случайное число 1-100</i>
<code>with lock:</code>	<i>with lock — захватываем замок</i>
<code>shared_data.append(num)</code>	<i>Только один поток может быть здесь</i>
<code>print(f'Добавлено: {num}')</code>	
<code>time.sleep(0.5)</code>	<i>Пауза перед следующим числом</i>
<code>def consumer():</code>	<i>Поток-потребитель</i>
<code>while not stop_event.is_set():</code>	
<code>time.sleep(1)</code>	<i>Ждём 1 секунду</i>
<code>with lock:</code>	<i>Захватываем тот же замок</i>
<code>if shared_data:</code>	<i>Если есть что обработать</i>
<code>total = sum(shared_data)</code>	<i>Суммируем все числа</i>
<code>count = len(shared_data)</code>	
<code>shared_data.clear()</code>	<i>.clear() — очищаем список</i>
<code>print(f'Сумма {count} чисел: {total}')</code>	
<code># Запускаем потоки</code>	
<code>t_prod = threading.Thread(target=producer, daemon=True)</code>	<i>daemon=True — поток завершится вместе с программой</i>
<code>t_cons = threading.Thread(target=consumer, daemon=True)</code>	
<code>t_prod.start()</code>	
<code>t_cons.start()</code>	
<code>print('Работаем 5 секунд...')</code>	
<code>time.sleep(5)</code>	<i>Главный поток ждёт 5 секунд</i>
<code>stop_event.set()</code>	<i>.set() — посылаем сигнал остановки</i>
<code>print('Стоп!')</code>	

► **Результат:**

Работаем 5 секунд...
Добавлено: 42

```
Добавлено: 17
Сумма 2 чисел: 59
Добавлено: 83
Добавлено: 5
Сумма 2 чисел: 88
...
Стоп!
```

⚠ GIL — важно знать: Python имеет GIL (Global Interpreter Lock) — потоки не ускоряют вычисления на CPU (числодробилки). Они полезны только для I/O операций (сеть, файлы). Для CPU-задач используй multiprocessing.

□ Тест: Тема 21

? Вопрос: Что делает `t.join()`?

- А) Запускает поток
- Б) Блокирует выполнение до завершения потока `t` ← *правильный ответ*
- В) Останавливает поток
- Г) Соединяет два потока

? Вопрос: Зачем нужен `threading.Lock()`?

- А) Ускоряет потоки
- Б) Защищает общие данные от одновременного изменения несколькими потоками ← *правильный ответ*
- В) Создаёт новый поток
- Г) Останавливает все потоки

? Вопрос: Когда потоки НЕ дают ускорения в Python?

- А) При работе с файлами
- Б) При HTTP-запросах
- В) При тяжёлых вычислениях на CPU ← *правильный ответ*
- Г) При работе с базами данных

Тема 22: Тестирование кода (unittest)

Тесты — это код, который проверяет другой код. Профессиональный разработчик пишет тесты всегда.

Задание 22.1: Тестирование строковых функций

☆☆☆

Напиши функции:

- `palindrome(s)` — True если строка-палиндром (игнорируй регистр и пробелы)
- `word_count(s)` — количество слов
- `capitalize_words(s)` — каждое слово с заглавной буквы

Затем напиши `TestStringFunctions` с минимум 3 тестами на каждую функцию.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

<code>import unittest</code>	Встроенный модуль тестирования
<code># Тестируемые функции</code>	
<code>def palindrome(s):</code>	
<code> clean = s.lower().replace(' ', '')</code>	Нижний регистр, убираем пробелы
<code> return clean == clean[::-1]</code>	Сравниваем с обратной строкой
<code>def word_count(s):</code>	
<code> return len(s.split())</code>	<code>.split()</code> делит по пробелам, <code>len</code> считаем
<code>def capitalize_words(s):</code>	
<code> return ' '.join(w.capitalize() for w in s.split())</code>	<code>.capitalize()</code> — первая буква заглавная
<code>class</code> <code>TestStringFunctions(unittest.TestCase):</code>	Класс тестов наследует <code>TestCase</code>
<code># Тесты palindrome</code>	
<code>def test_palindrome_simple(self):</code>	Имя теста начинается с <code>test_</code>
<code>self.assertTrue(palindrome('racecar'))</code>	<code>assertTrue</code> — ожидаем <code>True</code>
<code>def test_palindrome_with_spaces(self):</code>	
<code> self.assertTrue(palindrome('А роза упала на лапу Азора'))</code>	Известный русский палиндром
<code>def test_palindrome_false(self):</code>	
<code>self.assertFalse(palindrome('python'))</code>	<code>assertFalse</code> — ожидаем <code>False</code>
<code># Тесты word_count</code>	
<code>def test_word_count_normal(self):</code>	

<code>self.assertEqual(word_count('hello world'), 2)</code>	<i>assertEqual — ожидаем конкретное значение</i>
<code>def test_word_count_extra_spaces(self):</code>	
<code>self.assertEqual(word_count(' a b c '), 3)</code>	<i>split() игнорирует лишние пробелы</i>
<code>def test_word_count_empty(self):</code>	
<code>self.assertEqual(word_count(''), 0)</code>	<i>Граничный случай: пустая строка</i>
<code># Тесты capitalize_words</code>	
<code>def test_capitalize_basic(self):</code>	
<code>self.assertEqual(</code>	
<code>capitalize_words('hello world'),</code>	
<code>'Hello World')</code>	<i>Ожидаемый результат</i>
<code>def</code>	
<code>test_capitalize_already_upper(self):</code>	
<code>self.assertEqual(</code>	
<code>capitalize_words('PYTHON CODE'),</code>	<i>Уже заглавные</i>
<code>'Python Code')</code>	<i>.capitalize() делает первую заглавной, остальные строчными</i>
<code>if __name__ == '__main__':</code>	<i>Запускаем тесты при прямом запуске файла</i>
<code>unittest.main(verbosity=2)</code>	<i>verbosity=2 — подробный вывод</i>

► Результат:

```
test_capitalize_already_upper ... ok
test_capitalize_basic ... ok
test_palindrome_false ... ok
test_palindrome_simple ... ok
test_palindrome with spaces ... ok
test_word_count_empty ... ok
test_word_count_extra_spaces ... ok
test_word_count_normal ... ok
-----
Ran 8 tests in 0.001s
OK
```

⊖ Частая ошибка:

✗ `def test_check(): assert palindrome('aba')`

✓ `class TestFoo(unittest.TestCase): def test_check(self): self.assertTrue(palindrome('aba'))`

unittest требует: 1) класс, наследующий TestCase, 2) методы начинающиеся с test_. Голые assert-ы не интегрируются с unittest runner.

Задание 22.2: Тест с setUp и tearDown

★★★

Напиши тесты для класса BankAccount (баланс, deposit, withdraw, transfer).
Используй setUp() для создания тестового аккаунта перед каждым тестом.
Используй tearDown() для очистки.
Обязательно тестируй: нормальные случаи, граничные (0, отрицательные),
исключения.

✓ РЕШЕНИЕ С ПОЯСНЕНИЯМИ

import unittest	
class BankAccount:	Тестируемый класс
def __init__(self, owner, balance=0):	
self.owner = owner	
self.balance = balance	
def deposit(self, amount):	
if amount <= 0:	
raise ValueError('Сумма должна быть > 0')	
self.balance += amount	
def withdraw(self, amount):	
if amount <= 0:	
raise ValueError('Сумма должна быть > 0')	
if amount > self.balance:	
raise ValueError('Недостаточно средств')	
self.balance -= amount	
class TestBankAccount(unittest.TestCase):	
def setUp(self):	Вызывается ПЕРЕД каждым тестом
self.acc = BankAccount('Тест', 1000)	Свежий аккаунт для каждого теста
def tearDown(self):	Вызывается ПОСЛЕ каждого теста
del self.acc	Очищаем объект
def test_initial_balance(self):	Начальный баланс
self.assertEqual(self.acc.balance, 1000)	
def test_deposit_normal(self):	Нормальное пополнение

self.acc.deposit(500)	
self.assertEqual(self.acc.balance, 1500)	
def test_deposit_zero_raises(self):	Нулевой депозит — должна быть ошибка
with self.assertRaises(ValueError):	assertRaises проверяет исключение
self.acc.deposit(0)	Ожидаем ValueError
def test_withdraw_normal(self):	
self.acc.withdraw(300)	
self.assertEqual(self.acc.balance, 700)	
def test_withdraw_insufficient(self):	Снятие больше баланса
with self.assertRaises(ValueError):	
self.acc.withdraw(2000)	
def test withdraw exact balance(self):	Снятие ровно всего баланса
self.acc.withdraw(1000)	Граничный случай
self.assertEqual(self.acc.balance, 0)	Баланс стал ноль — это ок
if __name__ == '__main__':	
unittest.main(verbosity=2)	

► **Результат:**

```
test_deposit_normal ... ok
test_deposit_zero_raises ... ok
test_initial_balance ... ok
test_withdraw_exact_balance ... ok
test_withdraw_insufficient ... ok
test_withdraw_normal ... ok
-----
Ran 6 tests in 0.001s
OK
```

□ Тест: Тема 22

? Вопрос: Что делает setUp() в TestCase?

- А) Запускает все тесты
- Б) Вызывается перед каждым тестом ← *правильный ответ*
- В) Инициализирует модуль
- Г) Вызывается один раз перед классом

? Вопрос: Как проверить, что функция вызывает исключение?

- А) try/except в тесте
- Б) self.assertRaises(ErrorType) ← *правильный ответ*
- В) self.assertError()
- Г) self.checkException()

? Вопрос: Что означает 'OK' в финальном выводе unittest?

- А) Один тест прошёл
- Б) Все тесты прошли успешно ← *правильный ответ*
- В) Тесты не запускались
- Г) Найдены предупреждения

□ Итоговая таблица прогресса

Тема	Заданий	Тест	Отметка
Тема 15: Генераторы	3	3 вопр.	_____
Тема 16: Декораторы	2	2 вопр.	_____
Тема 17: Алгоритмы	2	2 вопр.	_____
Тема 18: CSV и JSON	2	2 вопр.	_____
Тема 19: requests (интернет)	3	3 вопр.	_____
Тема 20: SQLite	2	2 вопр.	_____
Тема 21: Многопоточность	3	3 вопр.	_____
Тема 22: Тестирование	2	3 вопр.	_____

□ **Часть 3 пройдена!** Всего: 13 заданий и 14 вопросов теста. Ты освоил профессиональные инструменты Python-разработчика. Следующий шаг — Часть 4: веб, данные, Telegram-боты и GUI!